

Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información
CI4321 – Computación Gráfica I
Sep–Dic 2026

Profesor: Jorge Palacios
Equipo: Sudo Render
Estudiantes: Baudilio Velasquez – 18-10665
Luis Isea – 19-10175

Fase 01: Apuntar al objetivo

Informe de implementación y pruebas

---

1. Introducción

Partimos del laboratorio del cubo con `FixedFunction` para construir un pequeño juego de lanzamiento de esferas en 3D. La idea era tener una torreta sobre un campo plano, colocar tres objetivos a distintas distancias y alturas, y permitir que el jugador ajustara la posición lateral, la inclinación del cañón y la fuerza antes de disparar. En esta fase nos interesó que no fuera solamente una escena para observar, sino que cada disparo respondiera a esos controles.

Trabajamos con TypeScript, WebGL2 y `gl-matrix`. Mantuvimos la biblioteca `FixedFunction` sin modificaciones y generamos las figuras con código. Al terminar la mecánica principal añadimos un marcador, mensajes de estado y tres dificultades para practicar con blancos fijos o en movimiento.

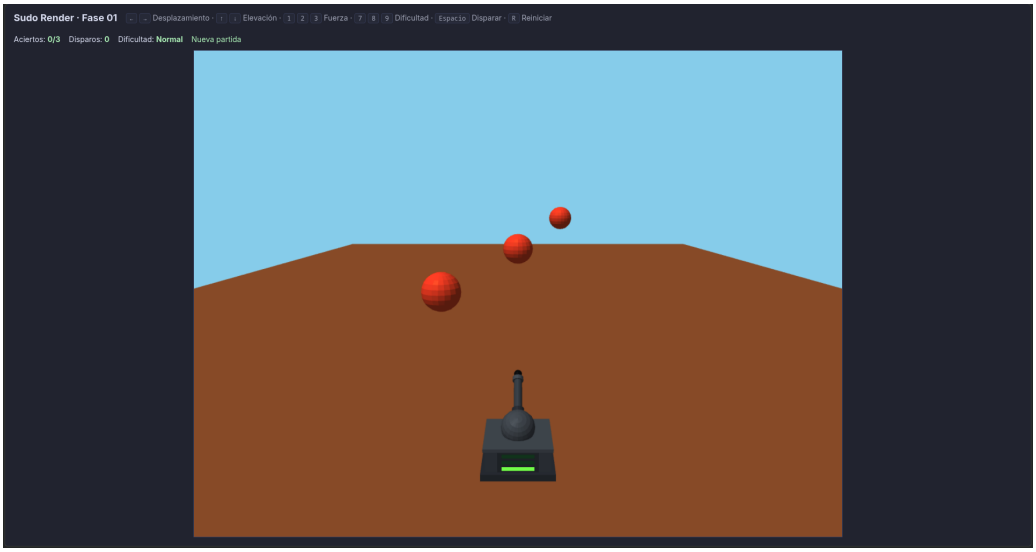


Figura 1: Vista general del juego al comenzar: torreta, terreno, tres objetivos y controles.

2. Construcción de la escena y la torreta

Organizamos el proyecto en módulos para separar los datos del juego, la entrada del teclado, la creación de las figuras y su dibujo. El renderizado utiliza una cámara fija en perspectiva, situada detrás y por encima de la torreta. Esto nos permite ver el recorrido lateral del cañón y los tres objetivos sin tener que controlar también la cámara.

El suelo es un plano marrón y el fondo tiene un tono azul cielo. Para la torreta combinamos prismas, una esfera que funciona como unión y un tubo cilíndrico. El cuerpo se desplaza horizontalmente, mientras que la unión y el cañón comparten la transformación de elevación. Por eso el tubo puede inclinarse sin que la base empiece a girar o avanzar. La esfera negra del proyectil permanece sobre la boca del cañón mientras está disponible.



Figura 2: Detalles del cañón, del indicador de fuerza y de los objetivos a diferentes profundidades.

3. Controles del jugador

Con las flechas izquierda y derecha movemos la torreta entre -8 y 8 unidades. Las flechas arriba y abajo ajustan la elevación del tubo entre 5° y 90°. Las teclas `1`, `2` y `3` seleccionan la fuerza, y las tres barras de la torreta muestran visualmente cuántos niveles están activos. Reservamos `Espacio` para disparar y `R` para reiniciar la partida. Los controles también aparecen en la parte superior de la página.

4. Movimiento de la bala y colisiones

Para lanzar la esfera calculamos su posición inicial a partir de la boca del cañón y de la inclinación seleccionada. Luego determinamos una velocidad que apunta hacia delante y hacia arriba. Los tres niveles de fuerza corresponden a velocidades iniciales de 12, 17 y 22 unidades por segundo; la gravedad utilizada es de 9,8 unidades por segundo al cuadrado.

En lugar de aumentar la posición de la bala una distancia fija en cada fotograma, calculamos su trayectoria a partir del tiempo que ha transcurrido desde el disparo. De ese modo el proyectil describe una parábola sin depender directamente del número de veces que se dibuja la escena. También mantenemos una sola esfera en vuelo: mientras el disparo está activo, pulsar nuevamente `Espacio` no crea otro proyectil.

Para detectar impactos revisamos el recorrido de la bala entre dos instantes consecutivos. Comparamos su distancia con el centro de cada objetivo, teniendo en cuenta el radio de ambos cuerpos. Si llega a tocar una esfera, esa esfera desaparece; si no acierta, el disparo termina al tocar el suelo. En ambos casos la bala vuelve a quedar disponible en el cañón. Además, mover la torreta mientras el proyectil está en el aire no altera la trayectoria que ya había comenzado.



Figura 3: Momento de un disparo: la bala se separó de la torreta y se acerca a un objetivo.

5. Marcador y final de partida

Para que fuera más sencillo saber cómo iba la partida, agregamos un marcador sobre el lienzo. Allí aparecen los aciertos sobre tres, el número de disparos realizados, la dificultad seleccionada y un mensaje breve con lo que acaba de suceder. Los disparos fallidos también suman al contador de intentos. Cuando alcanzamos las tres esferas mostramos un mensaje de felicitación que indica cuántos disparos necesitamos.

También registramos en la consola eventos concretos mediante `console.info`: cuando sale una bala, cuando llega al suelo, cuando alcanza un objetivo, cuando queda disponible otra vez y cuando reiniciamos o ganamos. Preferimos no escribir información en cada fotograma para que la consola siga siendo fácil de revisar. Durante nuestras comprobaciones no observamos errores ni advertencias.



Figura 4: Progreso que observamos: un objetivo alcanzado, dos objetivos y finalmente tres aciertos con el mensaje de victoria.

6. Tres niveles de dificultad

Además del modo original, incorporamos dos variantes en las que los objetivos se mueven de forma horizontal. Elegimos las teclas `7`, `8` y `9` para cambiar entre Normal, Difícil y Extremo, respectivamente. En Normal las esferas permanecen quietas; en Difícil se desplazan suavemente a izquierda y derecha; en Extremo hacen ese mismo movimiento con mayor rapidez, pero sin llegar a ser excesivo.

El desplazamiento se calcula como una oscilación que depende del tiempo de juego. Conservamos las alturas y profundidades de cada objetivo y modificamos su posición horizontal. Al implementar esto tuvimos que ajustar también las colisiones: no bastaba con mover las esferas visualmente, porque el programa tenía que detectar los impactos en sus posiciones actuales durante el vuelo. Al cambiar de dificultad, los blancos empiezan otra vez el movimiento desde sus posiciones originales.

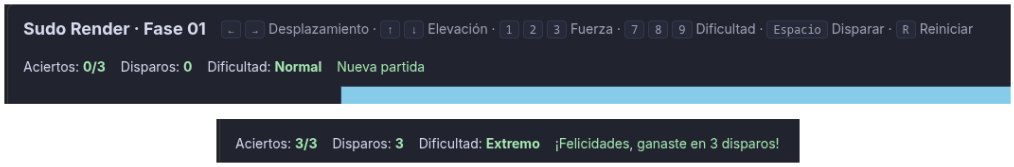


Figura 5: La interfaz informa la dificultad activa. Estas capturas muestran Normal y Extremo; el movimiento se aprecia al ejecutar el juego.

7. Pruebas y comprobaciones

Probamos el juego de principio a fin. En Normal alcanzamos los tres objetivos ajustando el desplazamiento lateral, la elevación y la fuerza. También hicimos disparos deliberadamente fallidos para comprobar que la bala regresara al cañón, y presionamos `R` tanto durante un vuelo como después de eliminar objetivos. En todos esos casos observamos que se restauraron las esferas, la posición inicial, la fuerza, el proyectil y el contador de la partida.

Para revisar la dependencia de los fotogramas utilizamos `npm run test:fps`. Esta prueba simula pasos de tiempo de 1/30 y 1/144 segundos sobre la lógica de física y colisiones y compara los resultados en los tres modos de dificultad. La comparación nos permite comprobar que los impactos evaluados no cambian por utilizar esas dos frecuencias de actualización. Es importante distinguir esto de una medición real del rendimiento gráfico: la prueba no cambia la frecuencia del monitor ni mide los FPS que alcanza el navegador.

Finalmente, ejecutamos la compilación del proyecto y revisamos la consola del navegador sin encontrar errores ni advertencias. Conservamos la generación de primitivas en código y no modificamos los archivos de `FixedFunction` proporcionados en el laboratorio.

En particular, revisamos varios casos que podían causar problemas si la lógica de la partida no mantenía correctamente su estado:

Comprobación	**Resultado observado**
Dos disparos seguidos	No apareció una segunda bala mientras la primera estaba en vuelo.
Disparo fallido	La bala regresó al cañón y pudimos intentarlo nuevamente.
Objetivo alcanzado	Desapareció la esfera correspondiente y aumentó el contador.
Reinicio durante el vuelo	Se canceló el disparo y reaparecieron los tres objetivos.
Victoria y dificultad	Se mostró el resultado de la partida y pudimos seleccionar otro modo.

8. Conclusión

Esta fase nos permitió pasar de construir objetos tridimensionales a hacerlos interactuar entre sí. La parte que más nos ayudó a comprender el problema fue relacionar la inclinación, la fuerza y el tiempo con el movimiento real de la bala, y después extender las colisiones a objetivos que ya no están quietos. El marcador y las dificultades no eran necesarios para explicar la trayectoria, pero hicieron que pudiéramos practicar y comprobar mejor cada disparo. Con las pruebas realizadas dejamos una partida que se puede completar y reiniciar sin perder el estado correcto de la escena.

Repositorio: `https://github.com/baudiliovm/sudo_render_fase01_ci4321`

Video demostrativo: `https://youtu.be/NYqM-H1vBHU`